# KnowOps AI: Phase 2.5 Technical Report

**Component:** User API, JWT Authentication & Web Operations Console
**Sprint:** sprint-1-(base) / Phase 2 | **Date:** October 2026 | **Status:** Completed & Verified

## 1. Executive Summary & Purpose

Phase 2.5 marks the **culmination of Phase 2**. It transforms the underlying AI and vector retrieval engines into a complete, user-facing operations platform. Phase 2.5 introduces a secure **JWT Authentication Layer**, a **Multi-Turn Conversation Memory System** backed by PostgreSQL, and an interactive **Web Operations Console (SPA)** with real-time health monitors, runbook quick links, and citation viewers.

**The Operational Analogy:**
- **Phase 2.1:** Built the AI reasoning engine (Gemini LLM & vector store).
- **Phase 2.2:** Ingested 61 operational runbook chunks.
- **Phase 2.3:** Built the pgvector HNSW retrieval engine.
- **Phase 2.4:** Verified search precision (91.7% Hit@1, 100% Hit@3, 0.958 MRR).
- **Phase 2.5 (This Phase):** The **Modern Command Center & Access Gate**. Provides on-call SREs with secure login credentials, full conversation history so the AI remembers context across follow-up questions, and an operations dashboard for fast incident triage.

## 2. Core Deliverables Built in Phase 2.5

| Component | Files / Artifacts | Key Responsibilities & Architecture |
|---|---|---|
| **Security & JWT Auth Layer** | app/auth.py | • PBKDF2-SHA256 password hashing (100,000 iterations + 16-byte salt)<br>• Constant-time credential verification via secrets.compare_digest<br>• Signed JWT token issuance and decoding using PyJWT and HMAC-SHA256<br>• FastAPI security dependencies: get_current_user_optional and get_current_user_required |
| **Conversation & Memory API** | app/main.py<br>app/schemas.py | • POST /conversations: Session thread creation in Neon PostgreSQL<br>• GET /conversations & GET /conversations/{id}: History retrieval<br>• POST /ask: Automatically loads prior conversation turns into the Gemini LLM context, enabling multi-turn question answering and follow-ups |
| **Web Operations Console (SPA)** | static/index.html<br>static/css/app.css<br>static/js/app.js | • Single-Page Application served at GET /<br>• Glassmorphism dark mode with live PostgreSQL, chunk count, and Gemini status pills<br>• Quick runbook cards (Disaster Recovery, K8s Ops, API Troubleshooting, On-Call)<br>• Dynamic prompt chips, copyable code blocks, expandable citation drawers<br>• Operator login modal and real-time system diagnostics drawer |
| **Automated Test Suites** | tests/test_auth.py<br>tests/test_api_v2_5.py | • 10 passing automated tests covering password hashing, credential checks, JWT tokens, expired token rejection, protected routes, and conversation lifecycle |

## 3. Multi-Turn Conversation Memory Workflow

A critical limitation of basic RAG systems is lack of memory between questions. In Phase 2.5, KnowOps AI maintains state across turns using PostgreSQL:

- **Turn 1 (New Incident):** Operator asks: *'What are the steps to detect primary database failure?'*. The system creates session #id in the conversations table, performs RAG retrieval, returns the answer with citations, and writes both query and answer into the messages table.
- **Turn 2 (Follow-up Question):** Operator asks: *'What command do I run to check if PostgreSQL is accepting connections?'*, passing conversation_id=#id.
- **History Injection:** The backend queries the messages table for the past conversation turns, constructs a multi-turn history list, and passes it to Gemini chat_completion(..., history=history).
- **Contextual Continuity:** The LLM accurately resolves pronouns, references previous steps, and continues the conversation naturally.

## 4. Security & Authentication Architecture

The authentication framework in app/auth.py provides enterprise security best practices:

| Security Aspect | Implementation Specification |
|---|---|
| **Password Hashing** | PBKDF2-HMAC-SHA256 with 100,000 iterations and per-password cryptographic 16-byte salt. |
| **Constant-Time Comparison** | secrets.compare_digest prevents timing attacks during authentication. |
| **Token Format & Algorithm** | Signed JSON Web Tokens (PyJWT) using HMAC-SHA256 (HS256) with 43-byte key. |
| **Standard Claims** | sub (username), role (operator/admin), exp (expiration), iat (issued at), iss (knowops-ai). |
| **Token Expiration** | Configurable via JWT_EXPIRE_MINUTES (default 60 minutes). Expired tokens return HTTP 401. |
| **FastAPI Protection** | HTTPBearer dependency extraction supports Swagger UI and Bearer header authentication. |

## 5. Verification & Automated Test Results

Phase 2.5 was verified with 10 dedicated automated test cases across two test modules:

**Phase 2.5 Test Results — 100% Pass Rate:**
- tests/test_auth.py::test_password_hashing → **PASSED**
- tests/test_auth.py::test_operator_credentials_verification → **PASSED**
- tests/test_auth.py::test_jwt_token_creation_and_decoding → **PASSED**
- tests/test_auth.py::test_expired_jwt_token_raises_401 → **PASSED**
- tests/test_auth.py::test_login_endpoint_success → **PASSED**
- tests/test_auth.py::test_login_endpoint_invalid_credentials → **PASSED**
- tests/test_auth.py::test_auth_me_endpoint_protected → **PASSED**
- tests/test_api_v2_5.py::test_serve_frontend_dashboard → **PASSED**
- tests/test_api_v2_5.py::test_conversation_lifecycle → **PASSED**
- tests/test_api_v2_5.py::test_conversation_not_found → **PASSED**

## 6. Phase 2 Complete Architectural Milestone Recap

With Phase 2.5 completed, the entire Phase 2 milestone is 100% operational:

| Sub-Phase | Deliverable | Status |
|---|---|---|
| **Phase 2.1** | Gemini AI Core Integration (app/llm.py) & Neon pgvector Setup | **COMPLETE** |
| **Phase 2.2** | Knowledge Base Ingestion (61 chunks loaded into HNSW index) | **COMPLETE** |
| **Phase 2.3** | RAG Semantic Retrieval Engine (app/rag.py, app/main.py) | **COMPLETE** |
| **Phase 2.4** | RAG Evaluation & Benchmarking (100% Hit@3, 0.958 MRR) | **COMPLETE** |
| **Phase 2.5** | User API, JWT Authentication & Web Operations Console SPA | **COMPLETE** |

## 7. Next Steps: Roadmap to Phase 3

With the core RAG platform, REST API, authentication, and frontend operational, the project is ready for **Phase 3 — Autonomous Support Agents & Workflow Automation**:

- **Phase 3.1 — Support Ticket CRM:** Mocked support ticket database and CRM endpoints (/tickets).
- **Phase 3.2 — Autonomous Agent Orchestrator:** Function-calling agent with tool execution for diagnostic lookups and ticket triage.
- **Phase 3.3 — Automated Runbook Actions:** Safe automated execution of verified runbook commands (e.g. pod restarts, scale checks).
- **Phase 3.4 — Cron & Workflow Automation:** Periodic re-indexing and incident webhook integrations.